

Contents

Contexto del proyecto — Estafeta (Tech Sphere Challenge 2026, Voice Agent Edition) **1**

- 1 · Qué es el reto 1
- 2 · Arquitectura del sistema — 4 capas 2
- 3 · Estructura de carpetas (dentro de la carpeta del proyecto, TechSource) 3
- 4 · Lo que se construyó en esta sesión: @techsphere/conversational 3
- 5 · Lo que se construyó en esta sesión: interfaz-conversacional/ (prototipo de prueba, voz a voz) 4
- 6 · Lo que se hizo en esta sesión — Paso 0 (recuperar el terreno) 6
- 7 · Lo que falta (según el plan heredado, Estafeta-Plan-de-Trabajo.md §6) 6
- 8 · Archivos clave para retomar, en orden de lectura sugerido 7
- 9 · Cómo verificar que algo está hecho (regla del propio proyecto) 7

Contexto del proyecto — Estafeta (Tech Sphere Challenge 2026, Voice Agent Edition)

Documento de traspaso para continuar este proyecto en otra conversación. Pégallo entero como primer mensaje si vas a trabajar con otro asistente de IA — está escrito para que alguien sin memoria de esta conversación pueda retomar sin perder contexto.

1 · Qué es el reto

Tech Sphere Challenge 2026 — edición Voice Agent, organizado por Source Meridian (con Pascual Bravo, AI Tinkerers Medellín, DB Crew LATAM, GDG Medellín, UNAL). Reto senior, individual, solo para personas en Colombia.

Qué hay que construir: un agente de voz con IA para seguimiento post-operatorio. Un paciente sale de una cirugía; en las horas siguientes el agente lo llama, conversa con él, entiende sus síntomas con información clínica real (RAG), y decide cuándo alertar a personal capacitado.

Premios: \$1.000 USD en créditos Claude prepagados (1º \$500, 2º \$300, 3º \$200) + posibilidad de entrar al pipeline de reclutamiento de Source Meridian para proyectos healthcare en EE. UU. Los 3 finalistas hacen demo en vivo el 5 de septiembre en Medellín.

Deadline: se corrió al **miércoles 12 de agosto de 2026, hasta la medianoche** (por el sismo del 10 de agosto). Verificar la fecha actual al retomar — puede haber cambiado de nuevo.

Entregables obligatorios (4): 1. Repositorio público en GitHub, con LICENSE MIT completo en la raíz (texto oficial completo, no solo el nombre del archivo). 2. Diagrama de arquitectura y del flujo de decisión. 3. Informe final con evidencia (prompts, configuraciones, capturas del demo). Sin esto, la entrega no se evalúa. 4. Video con demo funcionando + 2 preguntas respondidas frente a cámara: - P1: cómo venderías la solución a un cliente (problema, por qué esta solución, valor diferencial). - P2: la decisión técnica más relevante que tomaste — alternativas evaluadas, por qué las descartaste, riesgos, qué cambiarías con 2 semanas más.

5 compuertas eliminatorias (fallar una = la entrega no se puntúa): - Los 4 entregables completos. - La solución corre en ≤15 minutos siguiendo el README (credenciales incluidas). - Se documenta claramente qué modelo(s) se usó. - La conversación de voz en tiempo real funciona. - Subir y eliminar conocimiento desde la consola funciona: el agente aprende y olvida.

Rúbrica (100 pts): RAG/precisión clínica/conocimiento vivo pesa 20 pts; hay otros 5 criterios de 15-20 pts cada uno (repositorio del reto trae el detalle completo en docs/rubrica-evaluacion.md).

Stack: abierto, sin LLM único obligatorio. Hay un stack sugerido (Groq, Gemini, modelos locales, ChromaDB, Piper/Kokoro) pero “gana la ingeniería, no la billetera”.

2 · Arquitectura del sistema — 4 capas

Capa	Qué hace	Estado a la fecha de este documento
Interfaz de voz	Voz→texto y texto→voz. Nada más: no entiende, no extrae, no decide	Prototipo de prueba construido (ver §5)
Conversacional	El entrevistador. Modula tono/naturalidad/regionalismos(ver §4) Produce contexto estructurado	Implementada y probada
Determinista	Evalúa funcionalidad, interacción e integridad sobre contexto ya declarado suficiente. Aritmética pura, sin red	OK — Heredada, 93 pruebas en verde
Decisión	La única autoridad de decisión. Gobierna el bucle de suficiencia, compone los votos, emite la decisión y el resumen	OK — Heredada, 170 pruebas en verde

Cada capa desconoce la implementación de las otras y se comunica por **contratos tipados** (@techsphere/contracts) — es lo que permite que el sistema sobreviva a un cambio de modelo sin reescribirse.

La costura conversacional ↔ decisión (lo más importante para entender el sistema)

Vive entera en `contracts/src/conversational.ts`. Reglas clave:

- **La conversacional no juzga suficiencia global**, solo local (¿esta unidad quedó descrita?). La suficiencia global (¿ya alcanza para decidir?) es del decisor y solo del decisor.
- **El bucle:** el decisor responde `FrameVerdict` — o `sufficient` con la `Decision`, o `need_more` con un `frame_delta` (solo las unidades reabiertas).
- **El marco (ContextFrame) trae intent, no preguntas.** Prosa dirigida a la conversacional que dice QUÉ hay que saber, nunca CÓMO preguntarlo ni qué significa clínicamente. El criterio clínico vive entero del lado del decisor.
- **La conversacional no sabe en qué ronda va** (a propósito — evita que module su insistencia según el presupuesto del decisor).
- **La evidencia no se destruye** (ADR-004): `raw` guarda siempre el literal del paciente. El vacío se tipifica: `no_sabe` ≠ `no_comprende` ≠ `sin_respuesta` ≠ `rehusa`.
- **state (salud de la extracción, entero [-3,+3]) y confidence (fidelidad del mapeo, real [0,1]) son independientes** — ADR-005, no confundirlos.
- **La regla que no se negocia (ADR-024):** si el paciente no lo dijo, no existe. Expresiones que *tocan* una unidad sin cuantificarla (“calorcito”, “molestia”, “un poquito”) producen `normalized: null` con el `raw` intacto — **nunca** se traducen a un número o categoría inventada. Disparan un protocolo de reflejo (repreguntar con precisión), no una fabricación de dato.

Reglas que no se rediscuten (§8 del plan heredado)

1. **Un solo modelo para los dos roles de LLM** (entrevistador y decisor) — hace la compuerta G3 auditable con un `grep`.

2. **Ningún SDK de proveedor.** HTTP plano con fetch nativo de Node. npm ls no debe encontrar ningún cliente de terceros.
3. **Una sola constante de modelo por ruta**, en decision/src/modelo/rutas.ts (MODELOS_PERMITIDOS). El nombre no se repitea en ningún otro archivo del propio paquete decision; cualquier otro módulo que necesite el nombre lo **importa** de ahí, nunca lo hardcodea.
4. **La guarda de G3 corre al arrancar el proceso**, antes de pedir la credencial.
5. **El sistema no puede aceptar salida inválida** del modelo: toda salida estructurada cruza el validador de contratos, se reintenta acotado, y agotados los reintentos se degrada a humano.
6. **Ninguna sesión termina sin CallSummary.** Se ensambla desde el registro (ledger), nunca lo infiere el modelo.

Modelo vigente (ruta primaria): llama-3.3-70b-versatile en Groq (nube_groq), base <https://api.groq.com/openai/v1>. Hay una ruta de respaldo nube_google con gemini-3.5-flash. Temperaturas: decider = 0, interviewer = 0.4 (definidas en decision/src/modelo/rutas.ts).

3 · Estructura de carpetas (dentro de la carpeta del proyecto, TechSource)

```
TechSource/
├── conversacional.txt                (vacío, sin usar)
├── ParticipantArtifacts/            (kit oficial del reto, clonado de GitHub)
│   ├── dataset/                    (pacientes sintéticos colombianos, .xlsx)
│   ├── docs/rubrica-evaluacion.md
│   └── docs/stack-tecnico.md
├── Rescate/                         (el proyecto heredado que se está recuperando/completando)
│   ├── Estafeta-Plan-de-Trabajo.md ← documento de arranque original, LEER PRIMERO
│   ├── ParticipantArtifacts/        (copia del kit)
│   └── Herencia/                    (el código heredado + lo nuevo)
│       ├── README.md
│       ├── Especificacion-Capa-Decision.md      (64 KB, ADRs de decisión)
│       ├── Especificacion-Capa-Determinista.md   (32 KB, ADRs de determinista)
│       └── docs/ dominio/                      (creado en esta sesión, Paso 0)
│           ├── dominio-postop-v0.1.json
│           └── lexico-postop-v0.1.json
├── dominio/                         (ubicación original heredada, redundante con docs/dominio)
│   ├── corpus-texto/                 (documentos clínicos en texto plano)
│   ├── contracts/                    OK – 51 pruebas – tipos, puertos, validadores compartidos
│   ├── deterministic/               OK – 93 pruebas – evaluador estructural puro
│   ├── decision/                    OK – 170 pruebas – la única autoridad de decisión
│   ├── conversacional/              OK – 20 pruebas – CONSTRUIDO EN ESTA SESIÓN (ver §4)
│   └── interfaz-conversacional/ ← CONSTRUIDO EN ESTA SESIÓN (ver §5), prototipo de prueba de voz
```

4 · Lo que se construyó en esta sesión: @techsphere/conversational

Paquete completo en Rescate/Herencia/conversacional/, implementado contra contracts/src/conversation (la especificación efectiva, porque Especificacion-Capa-Conversacional.md no llegó en la copia heredada).

Archivos

- **src/motor-estados.ts** — el motor de estados puro (sin red, sin modelo). Tabla de transiciones, cierre por causa (CAUSAS_POR_CIERRE), selección del próximo acto (elegirActo), aplicación de extracciones (aplicarExtraccion) y de causas (aplicarCausa).

- **src/motor-nube.ts** — `crearMotorDeNube(opciones)`: la implementación real del puerto `ConversationalEngine` (`interpret + render`) contra Groq, HTTP plano, con validación de esquema local y reintentos acotados. El nombre del modelo **siempre** entra como parámetro, nunca hardcodeado aquí.
- **src/sesion.ts** — las cinco funciones públicas que el resto del sistema ya esperaba (nombres respetados tal cual, hay código heredado — `decision/scripts/muestra-estratificada.mjs` — que ya los importa):
 - `iniciarSesion(session_id)`
 - `cargarMarco(estado, frame)`
 - `conducirTurno(estado, textoPaciente, motor, turno)`
 - `cerrarPendientesPorCorte(estado, causa)`
 - `unidadesParaEntrega(estado)`
- **src/index.ts** — superficie pública del paquete.
- **test/** — 20 pruebas (`node --test`), cubren: motor de estados, la regla ADR-024 (nunca inventa un valor), cierre declarado vs. degradación con reintento, bandera roja, y una integración completa con un motor simulado (sin red) validando la salida contra `validateUnitResults` del módulo de contratos.

Decisiones de diseño importantes (por si hay que retomar o justificar en el informe)

- **Sentinela interno tocada: boolean** en cada unidad: `UnitExtraction` (el tipo del contrato) no tiene un valor para “todavía no se le preguntó” — sus 4 valores son todos estados de cierre o evidencia parcial. Sin este campo interno, una unidad recién creada (que por defecto tiene `extraction: "suspendida"`) era indistinguible de una que el motor ya cerró por agotamiento. Nunca cruza a `UnitResult` (se proyecta fuera en `unidadesParaEntrega`).
- **Tolerancia de reintento = 1** antes de que una causa de degradación (`no_comprende`, `incoherente`, `sin_respuesta`) cierre la unidad en `state: -3`. Las causas “declaradas” (`no_sabe`, `no_aplica`, `rehusa`) cierran de inmediato con `state +1` — son respuestas sanas, no fracasos (así lo dice el propio validador de `contracts`).
- **Validación de la salida del modelo:** como `@techsphere/contracts` solo publica (vía `package.json` → `exports`) los validadores de la *costura* (`ContextFrame`, `UnitResult`, `Decision`), no los de `EngineExtraction/EngineSignal` (que son forma interna del puerto `ConversationalEngine`, no cruzan la costura), se escribieron validadores locales mínimos en `motor-nube.ts`, con el mismo vocabulario de `ValidationIssue` que el resto del sistema.

Verificación

```
cd Rescate/Herencia/conversational
npm test          # 20/20 en verde
npm run build     # compila sin errores de tipos (tsc estricto, exactOptionalPropertyTypes)
```

Las 314 pruebas heredadas (`contracts` 51 + `deterministic` 93 + `decision` 170) también se dejaron en verde en esta sesión — ver §6, Paso 0.

5 · Lo que se construyó en esta sesión: interfaz-conversacional/ (prototipo de prueba, voz a voz)

Importante — alcance deliberadamente acotado: a pedido explícito del usuario, esta herramienta **prueba solo la capa conversacional**. NO invoca la capa de decisión (`Orquestador`, `DecisionEngineNube`), NO consulta el RAG, NO escala a nadie. Es un banco de pruebas de un solo tramo: paciente habla → agente interpreta y responde → paciente escucha.

Ubicación: `Rescate/Herencia/interfaz-conversacional/`.

Qué hace

- **Backend** (servidor.mjs, Node puro con node:http, sin frameworks): sirve una página estática y expone:
 - POST /api/transcribir — recibe audio grabado, lo manda a **Whisper large-v3 en Groq** (mismo API key, misma ruta de nube), devuelve el texto.
 - POST /api/turno — recibe texto del paciente, llama conducirTurno del paquete conversational, devuelve lo que dice el agente + el estado interno completo de cada unidad (para depurar).
 - POST /api/reiniciar — arranca sesión nueva. Usa buildFrameGenerico de @techsphere/decision **solo como generador de estructura** (las 6 unidades reales del dominio — fiebre, dolor, herida, movilidad, apetito, sueño — con su léxico y regionalismos reales, cero criterio clínico) para tener sobre qué conversar sin inventar un marco a mano.
- **Frontend** (public/index.html, un solo archivo, vanilla JS):
 - Botón de **mantener-presionado-para-hablar** (push-to-talk, vía MediaRecorder) — la salida segura que el plan recomienda para una demo en vivo.
 - El agente **habla en voz alta** las respuestas con speechSynthesis (voz del navegador/SO) — es lo que el plan del reto recomienda para el primer día, sin esperar a Piper/Kokoro. Se puede reemplazar después sin tocar el backend.
 - Panel de depuración en vivo: por cada unidad, muestra raw, normalized, state, confidence, cause, closure — para verificar a ojo que nunca se inventa un dato.
 - Mide y muestra la **latencia desde que el paciente deja de hablar hasta que el agente empieza a responder** — la métrica que más pesa según el plan.
 - Campo de texto como respaldo para depurar sin usar el micrófono.

Cómo correrla

```
cd Rescate/Herencia
cd contracts && npm install && npm run build && cd ..
cd deterministic && npm install && npm run build && cd ..
cd conversational && npm install && npm run build && cd ..
cd decision && npm install && npm run build && cd ..
cd interfaz-conversacional
npm install
GROQ_API_KEY=tu_clave npm start
```

Abrir <http://localhost:8787> en Chrome o Edge (mejor soporte de MediaRecorder y voces en español).

Nota de verificación honesta: el entorno donde se construyó esto no tenía salida a internet ni micrófono, así que no se pudo probar el tramo real de red (Whisper/Groq) ni el ciclo de voz completo. Sí se verificó que el servidor arranca, sirve la página, compila sin errores, y que /api/transcribir llega hasta Groq y responde con un error limpio (no un cuelgue) ante una clave inválida. **El primer round-trip de voz real hay que probarlo en la máquina de quien retome esto.**

Si algún npm install no encuentra un paquete hermano

Es la misma causa raíz que ya diagnosticó el plan heredado: los enlaces simbólicos de node_modules/@techsphere/* no sobreviven bien la sincronización de OneDrive. Volver a correr npm install en ese paquete puntual, con la carpeta hermana ya presente al lado, lo resuelve.

6 · Lo que se hizo en esta sesión — Paso 0 (recuperar el terreno)

Antes de construir nada, se aplicaron los dos arreglos de empaquetado que el plan heredado ya había diagnosticado:

1. **@techsphere/contracts no estaba enlazado** en decision/ y deterministic/ (symlinks rotos por la copia). Se resolvió con `npm install` en cada paquete, con contracts/ presente como carpeta hermana.
2. **El dominio estaba en la raíz** (Herencia/dominio/) y el código lo busca en Herencia/docs/dominio/. Se copiaron dominio-postop-v0.1.json y lexico-postop-v0.1.json al lugar correcto.

Resultado: **314/314 pruebas heredadas en verde** (contracts 51, deterministic 93, decision 170), y la demo de la compuerta G5 (`npm run demo:g5` en decision/) corriendo y mostrando el ciclo completo aprender→recuperar→retirar→olvidar en un solo proceso, sin reiniciar nada.

7 · Lo que falta (según el plan heredado, Estafeta-Plan-de-Trabajo.md §6)

El plan original define este orden de trabajo por compuertas eliminatorias:

- **Paso 0** — Hecho en esta sesión.
- **Paso 1** — La capa conversacional contra el contrato. Hecho en esta sesión (@techsphere/conversational).
- **Paso 2** — El prompt del entrevistador. Parcialmente cubierto: los prompts de sistema para interpret y render están escritos dentro de conversational/src/motor-nube.ts, en español colombiano coloquial, pero **no se auditaron línea por línea leyéndolos en voz alta** como recomienda el plan (§6 del documento de arranque original) — es tarea pendiente de refinamiento, especialmente las listas de sinónimos/requires_precision del léxico regional (aunque el léxico base ya viene poblado en docs/dominio/lexico-postop-v0.1.json con datos reales del corpus).
- **Paso 3** — La interfaz de voz “de verdad” (integrada al flujo completo, no solo el banco de pruebas del §5). Sigue **pendiente**:
 - Decidir entre la forma heredada (push-to-talk contra un contrato /turn, React+Vite+TS — recomendada) vs. LiveKit Agents (streaming continuo, mejor calidad pero requiere rearquitectura porque decisión está construida por turnos).
 - Reemplazar la voz del sistema operativo por **Piper** (local, latencia mínima, voces en español) o **Kokoro** (mejor calidad, más pesado) — el TTS de Groq NO sirve, solo habla inglés y árabe.
 - Pausas/silencios: decidir qué pasa mientras el agente “piensa” (sonido, muletilla, indicador) — un paciente asustado en 3 segundos de silencio cuelga.
 - Espera adaptativa al ritmo del paciente (no pausa fija: un postoperado habla despacio).
- **Paso 4** — Integración extremo a extremo: **Voz → conversacional → decisión → voz**, completa. Ahora mismo el banco de pruebas de voz (§5) para deliberadamente en el tramo conversacional; falta conectar con Orquestador/DecisionEngineNube de decision/ (que ya existen y están probados) para que el ciclo llegue hasta la Decision y el CallSummary real. Hay una sesión de referencia en decision/salidas/sesiones/ con el resultado esperado.
- **Paso 5** — Ensayo de la compuerta G2 en máquina limpia, cronometrado: alguien que no escribió el README debe poder levantar todo el sistema en ≤15 minutos.
- **Paso 6** — Los 4 entregables: repositorio, diagrama de arquitectura, informe final (con las 2 preguntas de cierre respondidas), y video con demo grabada.

Reparto sugerido originalmente (puede ya no aplicar si el equipo cambió)

Frente	Por qué
Recuperación del terreno y capa conversacional	Requiere leer el contrato y las specs heredadas — ya hecho
Prompt del entrevistador, léxico y regionalismos	El reto se evalúa oyendo la conversación; una voz que suena a doblaje neutro pierde puntos
Interfaz de voz	Pendiente de decisión de arquitectura (§Paso 3 arriba)
Ensayo de G2 y video	G2 se verifica cronometrado; el video pesa 15 puntos

8 · Archivos clave para retomar, en orden de lectura sugerido

1. Rescate/Estafeta-Plan-de-Trabajo.md — el plan de trabajo completo heredado, con más detalle que este resumen.
2. Rescate/Herencia/contracts/src/conversational.ts — la especificación efectiva de la costura conversacional↔decisión (densamente comentada).
3. Rescate/Herencia/conversational/ — lo construido en esta sesión (§4 de este documento).
4. Rescate/Herencia/interfaz-conversacional/ — el banco de pruebas de voz (§5 de este documento).
5. Rescate/Herencia/decision/README.md y Rescate/Herencia/decision/src/index.ts — para entender qué expone la capa de decisión y cómo cablearla al flujo completo (Paso 4).
6. Rescate/Herencia/Especificacion-Capa-Decision.md y Especificacion-Capa-Determinista.md — los ADR completos, material para la Pregunta 2 del video.

9 · Cómo verificar que algo está hecho (regla del propio proyecto)

“Se abre el artefacto, no se lee el informe.” Cuanto mejor razonado esté el reporte de lo que se hizo, más urgente es correr la prueba.

Una capa está hecha cuando sus pruebas pasan, no cuando su código existe. Antes de dar por buena cualquier continuación de este proyecto, correr:

```
cd Rescate/Herencia/contracts && npm test      # 51
cd ../deterministic && npm test                  # 93
cd ../decision && npm test                       # 170
cd ../conversational && npm test                 # 20
```

Total esperado: **334 pruebas en verde**. Si algo se rompe, esa es la referencia — no se avanza sobre rojo.